



**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Machine Learning (MLOps) Medication Side-Effect Prediction Using ML

USN

1RV23AI066

1RV23AI133

1RV23AI060

1RV23AI047

Name

Neelam J

Pranav pratyush

M.P Sahasra

Kumar Yash



Agenda

Introduction :

Medication safety is an important concern in healthcare, especially when patients take multiple medicines at the same time. Even though individual drugs may be safe, certain combinations can cause harmful drug interactions and early side effects.

Most existing systems detect these problems only after symptoms appear, which can delay treatment and increase health risks. Manual checking of drug interactions is also time-consuming and prone to error.

With the help of machine learning, it is possible to analyze drug data and predict potential interactions early. This project aims to build an AI-based system that identifies drug interactions and medication side effects in advance, improving patient safety and supporting healthcare professionals.

Motivation:

Drug interactions and medication side effects are a major cause of avoidable health complications. Many of these issues occur because potential risks are not identified early. Doctors and pharmacists often rely on manual checks or static databases, which can be slow and may miss complex interactions.

With the growing number of patients taking multiple medications, there is a strong need for an automated and intelligent system. This project is motivated by the goal of using Machine Learning to provide early warnings, improve medication safety, and reduce preventable medical errors.



Problem definition

Patients are often prescribed multiple medications to treat different health conditions. While these medicines may be safe individually, their combined use can lead to harmful drug interactions and early side effects.

Existing drug interaction checking methods are mostly manual, static, and not personalized based on patient characteristics such as age or body type. As a result, many adverse effects are detected only after they cause harm.

The problem is to design an intelligent system that can predict drug interactions and early medication side effects in advance using machine learning, thereby improving patient safety and supporting healthcare professionals.



Objectives

- To identify potential drug–drug interactions using Machine Learning techniques
- To predict early medication side effects before they cause serious harm
- To incorporate patient-specific factors such as age and body type
- To reduce medication-related errors and improve patient safety
- To support healthcare professionals with reliable decision support



Dataset Description (FAERS & SIDER)

FAERS Dataset (FDA Adverse Event Reporting System) : The FAERS dataset is maintained by the U.S. Food and Drug Administration. It contains real-world reports of adverse drug events submitted by healthcare professionals, manufacturers, and patients.

This dataset includes:

- Drug names involved in adverse events
- Reported side effects and reactions
- Patient demographic information (age group, gender, etc.)
- Severity of adverse events

FAERS provides real-world evidence of how drugs behave after being released into the market, making it valuable for detecting potential drug interactions and rare side effects.

SIDER Dataset (Side Effect Resource): The SIDER dataset contains information about drugs and their known side effects extracted from public documents and drug labels.

This dataset includes:

- Drug names
- Documented side effects
- Frequency and severity of side effects
- Drug–side effect associations

SIDER offers structured and reliable information about known side effects under normal usage conditions.



Risk Analysis Matrix

Scale Used: Probability: Rare (1), Unlikely (2), Moderate (3), Likely (4), Almost Certain (5).

Impact: Minor (1), Significant (2), Major (3), Severe (4), *Catastrophic* (5).

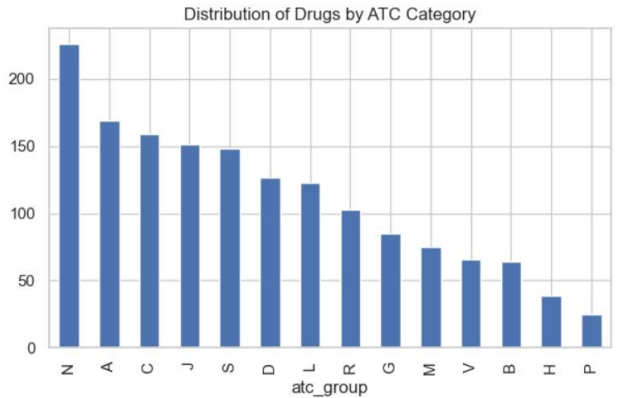
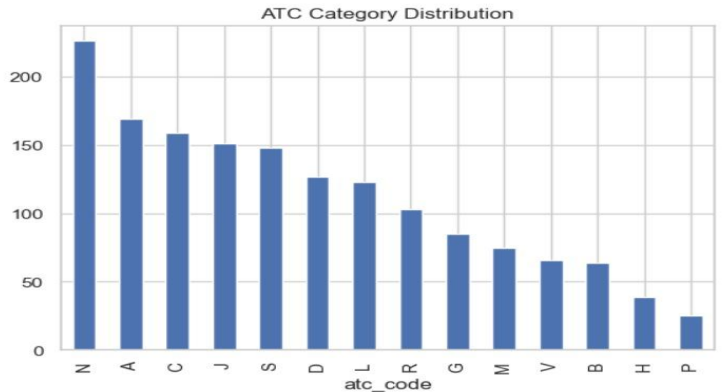
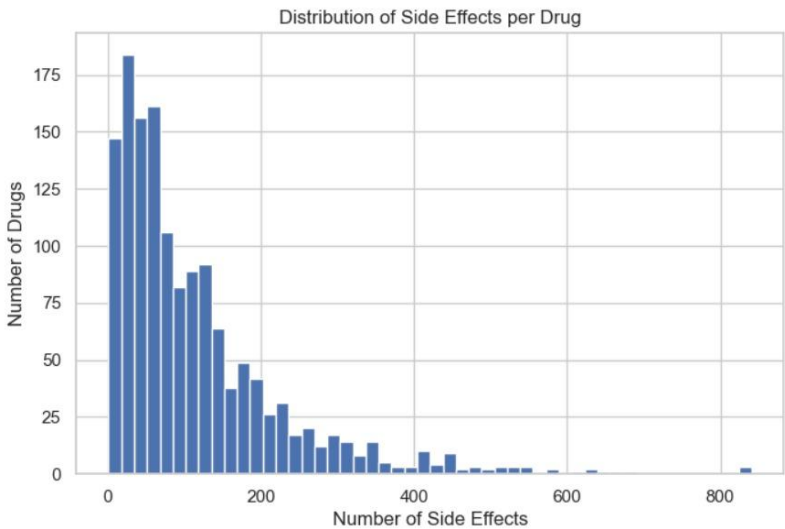
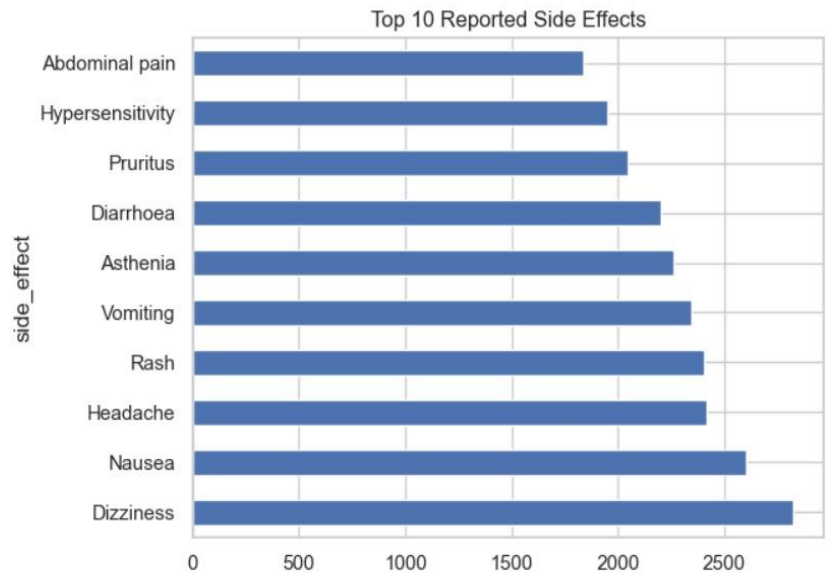
Severity = Probability × Impact

Risk ID	Risk Description	Probability	Impact	Severity Score	Risk Level
R1	Variation in prediction accuracy due to different body types	Almost Certain (5)	Major (4)	20	High
R2	Dependency on external drug databases (FAERS / SIDER updates)	Almost Certain (5)	Significant (3)	15	High
R3	Misinterpretation of alerts by users	Moderate (3)	Severe (5)	15	High
R4	Incomplete or inaccurate patient/drug data	Likely (4)	Significant (3)	12	Medium
R5	Incorrect drug name mapping	Unlikely (2)	Major (4)	8	Medium
R6	Bias in FAERS and SIDER datasets	Likely (4)	Significant (3)	12	Medium
R7	Data unavailability or delayed updates	Unlikely (2)	Severe (5)	10	Medium
R8	Skill loss due to over-reliance on automated predictions	Rare (1)	Minor (2)	2	Low
R9	Legal and ethical compliance issues	Moderate (3)	Severe (5)	15	High
R10	Incorrect voice input / bad pronunciations of drug names	Moderate (3)	Severe (5)	15	High
R11	Complex or confusing user interface	Likely (4)	Minor (2)	8	Medium



Exploratory Data Analysis (EDA)

	stitch_id	drug_name	atc_code	atc_group	stitch_flat	stitch_stereo	umls_id	meddra_type	umls_meddra_id	side_effect
0	CID100000085	carnitine	A16AA01	A	CID100000085	CID000010917	C0000729	LLT	C0000729	Abdominal cramps
1	CID100000085	carnitine	A16AA01	A	CID100000085	CID000010917	C0000729	PT	C0000737	Abdominal pain
2	CID100000085	carnitine	A16AA01	A	CID100000085	CID000010917	C0000737	LLT	C0000737	Abdominal pain





List of algorithms chosen to build ML model

Algorithms Considered

- **Logistic Regression**
 - Used as a baseline classification model.
 - Simple and interpretable, but limited in capturing complex non-linear relationships.
- **Decision Tree Classifier**
 - Captures non-linear decision boundaries.
 - Easy to interpret but prone to overfitting on complex datasets.
- **Random Forest Classifier (Selected Model)**
 - Ensemble-based algorithm combining multiple decision trees.
 - Handles non-linear relationships and feature interactions effectively.
 - Robust to noise and reduces overfitting.
 - Provided the best overall performance on evaluation metrics.



Metrics used to evaluate the ML model

Evaluation Metrics

- **Accuracy**
 - Measures the overall correctness of predictions.
 - Indicates how often the model predicts interaction risk correctly.
- **Precision**
 - Measures the proportion of correctly predicted interaction cases.
 - Important to reduce false positive drug interaction warnings.
- **Recall**
 - Measures the ability of the model to identify actual interaction cases.
 - Crucial in healthcare to avoid missing harmful drug interactions.
- **F1-Score**
 - Harmonic mean of Precision and Recall.
 - Used as the primary metric due to class imbalance in medical data.



Evaluation of ML model

- The model was evaluated using **Stratified 5-Fold Cross-Validation**
- Cross-validation ensures **robust and unbiased** performance estimation
- Evaluation was performed using multiple metrics to capture overall performance
- The model achieves **high accuracy**, indicating strong overall prediction capability
- Precision indicates the model's ability to correctly identify true interactions
- Recall reflects conservative prediction behavior, prioritizing precision
- F1-Score provides a balanced measure between precision and recall
- Results demonstrate the model's suitability for real-world deployment

Metric	Score
Accuracy	0.958
Precision	0.726
Recall	0.246
F1-Score	0.365



Identification of Best Model

Identification of Best Model

- Multiple algorithms were initially considered for model development
- **Random Forest Classifier** was selected as the final model
- The model provides **high accuracy with stable performance across folds**
- Random Forest handles **feature interactions and non-linear relationships effectively**
- Demonstrates robustness to noise and overfitting through ensemble learning
- Produces consistent results suitable for deployment and monitoring
- Selected as the best model based on **overall evaluation metrics and stability**



Phase II – MLOPs Life Cycle

	Component	MLOPs Tools
1	Data Management	DVC is used to version and manage FAERS and SIDER datasets. It ensures data consistency and reproducibility across experiments.
2	Experiment Tracking and Version Control	ML FLOW : MLflow is an MLOps tool used to track, manage, and compare machine learning experiments .
3	Automation and Pipeline Orchestration	GitHub Actions / Python pipelines automate preprocessing, training, and evaluation. This ensures repeatable and error-free ML workflows.
4	Model Deployment & Serving	Docker with Flask/FastAPI is used to deploy the trained model as a service. It enables real-time prediction of drug interaction risks.
5	Monitoring & Governance and collaboration	MLflow and GitHub monitor model performance and manage versions. They support collaboration, traceability, and compliance.



Phase II – MLOPs Life Cycle

DATA MANAGEMENT :

DVC (Data versioning control)In Machine Learning projects, **data changes frequently** due to preprocessing, feature engineering, or updates.

Traditional Git is **not suitable for tracking large datasets**, as it increases repository size and slows down version control.

DVC is used to solve this problem by tracking data versions without storing the actual data in Git.

Step 1: dvc status

```
: \Users\neela>cd C:\Users\neela\Documents\drug-interaction-mlops  
  
: \Users\neela\Documents\drug-interaction-mlops>python -m dvc status  
Data and pipelines are up to date.  
  
: \Users\neela\Documents\drug-interaction-mlops>
```

Here, DVC checks whether the tracked dataset has changed compared to the last committed version.

The message *“Data and pipelines are up to date”* confirms that the dataset used for training is exactly the same as the version recorded by DVC.

This ensures data consistency and reproducibility across experiments.



Step 2 :

```
C:\Users\neela\Documents\drug-interaction-mlops>python -m dvc status
Data and pipelines are up to date.

C:\Users\neela\Documents\drug-interaction-mlops>python -m dvc list . data
training_data_pairs.pkl
training_data_pairs.pkl.dvc

C:\Users\neela\Documents\drug-interaction-mlops>
```

This command shows that the training dataset is tracked by DVC using a **.dvc** file instead of being stored directly in Git.

The actual dataset remains locally, while Git tracks only the metadata file generated by DVC.

Key point to highlight

- Large datasets are **not pushed to Git**
- Only metadata is versioned
- Keeps repository lightweight



```
C:\Users\neela\Documents\drug-interaction-mlops>python -m dvc dag
WARNING: Unable to find 'less' in the PATH. Check out <https://man.dvc.org/pipeline/show> for more info.
+-----+
| data\training_data_pairs.pkl.dvc |
+-----+

C:\Users\neela\Documents\drug-interaction-mlops>
```

The DAG (Directed Acyclic Graph) shows the relationship between data artifacts in the pipeline.

Here, the training dataset is represented as a tracked output, which forms the base of the machine learning pipeline.

Step 3

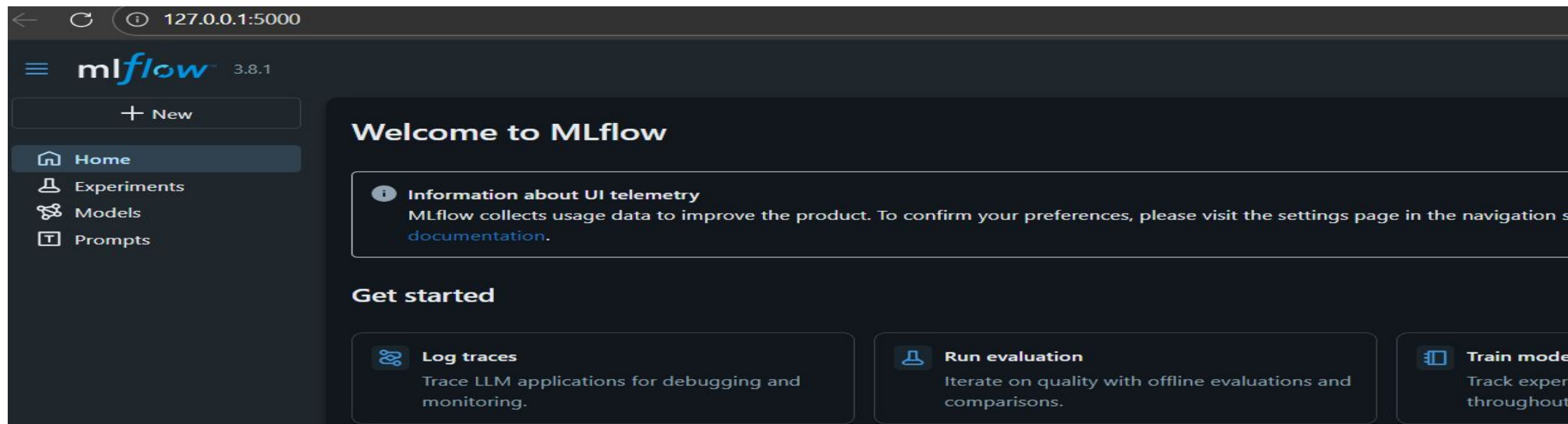


Phase II – MLOPs Life Cycle

Experiment Tracking and Version Control :

ML flow :

Step 1:



This screen shows the MLflow Tracking Server dashboard, where all machine learning experiments are organized. It provides a centralized interface to manage experiments, runs, metrics, and models.

Why it is important

This confirms that MLflow is correctly set up and running. It ensures that all experiments are tracked in one place instead of relying on manual logs or console outputs, which is a core requirement in MLOps.



ML flow procedure

Step 2 :

The screenshot displays the MLflow Tracking Server dashboard (version 3.8.1) for a run named 'RandomForest_Drug_Interaction'. The interface includes a navigation bar with 'Overview', 'Model metrics', 'System metrics', 'Traces', and 'Artifacts'. The 'Overview' tab is active, showing a 'Description' section with 'No description' and a 'Metrics (2)' table. The 'About this run' section on the right provides details about the run's creation, status, and source. The 'Parameters (3)' section is partially visible at the bottom.

Metric	Value
accuracy	0.5233333333333333
f1_score	0.5119453924914675

About this run	
Created at	01/03/2026, 08:13:30 PM
Created by	neela
Experiment ID	0
Status	Finished
Run ID	e88d002db4244b8b951f2f81f10fe593
Duration	265ms
Source	train_model.py 3be482c
Logged models	
Registered prompts	

Parameters (3)	

This screen shows the MLflow Tracking Server dashboard, where all machine learning experiments are organized. It provides a centralized interface to manage experiments, runs, metrics, and models.

Why it is important : This confirms that MLflow is correctly set up and running. It ensures that all experiments are tracked in one place instead of relying on manual logs or console outputs, which is a core requirement in MLOps.



Mlflow procedure

Step 3 :



This screen lists all experiments created in MLflow. Each experiment groups multiple training runs under a single project context.

Why it is important Organizing runs into experiments allows systematic comparison of models trained on the same problem. It avoids confusion when multiple training attempts are performed.



Phase II – MLOPs Life Cycle

Automation and Pipeline Orchestration

Implemented an automated ML workflow using **Kuberflow Pipelines**

Designed modular pipeline components for:

- Data loading
- Data preprocessing
- Model training
- Model evaluation
- Model registration

Pipeline components are written in **Python** for reusability

End-to-end workflow compiled into a **YAML pipeline definition**

Enables **repeatable and reproducible** ML experiments

Reduces manual intervention in model training and evaluation

Integrated with **GitHub Actions** for automated execution on code changes



Phase II – MLOPs Life Cycle

Model Deployment & Serving

- The trained **Random Forest model** is deployed as a web service using **FastAPI**
- **Docker** is used to containerize the application for portability and consistency
- The deployment exposes REST APIs for real-time inference
- Fast API enables lightweight, high-performance model serving
- Swagger UI is used for easy API testing and documentation
- The deployed service allows **real-time prediction of drug interaction risks**
- The solution is scalable and suitable for production deployment



Phase II – MLOPs Life Cycle

Response body

```
{
  "drug_1": "aspirin",
  "drug_2": "warfarin",
  "risk_level": "HIGH RISK",
  "confidence_score": 0.95,
  "confidence_level": "Very High",
  "explanation": "Known clinical interaction based on shared therapeutic class, side-effect overlap, or co-prescription risk.",
  "drift_detected": false
}
```



Download

Response headers

```
content-length: 267
content-type: application/json
date: Fri, 09 Jan 2026 09:46:11 GMT
server: uvicorn
```

default

GET /metrics Metrics

GET / Health

GET /drugs Get Drugs

POST /predict Predict

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "drug_name_1": "aspirin",
  "drug_name_2": "warfarin"
}
```



Phase II – MLOPs Life Cycle

Monitoring, Governance & Collaboration

- **Docker** ensures consistent and reproducible deployment across environments
- **GitHub Actions (CI/CD)** automates build, testing, and pipeline execution on code changes
- **MLflow** is used to track model experiments, metrics, and saved artifacts
- **Prometheus** collects real-time inference metrics from the deployed API
- **Grafana** visualizes model performance metrics such as prediction request count
- **Version control using GitHub** ensures traceability of code, data, and model changes
- Enables **collaboration** among team members through shared repositories and workflows
- Provides **governance** through monitoring, logging, and controlled model lifecycle management

Phase II – MLOPs Life Cycle

Grafana Dashboard Showing Real-Time Prediction Request Monitoring

